MACHINE LEARNING CAPSTONE · 2026

Ranking Content by Observed Decline Risk

A machine-learning approach to prioritizing content for human review

Nikita Sudarshan
Machine Learning Capstone · 2026

Metric	Value
Test observations	**6,163**
Test-set down rate	**52.5%**
Precision@500	**61.2%**
Lift vs baseline	**2.35×**

Abstract

- Which content pages should a team review first when many pages show different levels of performance?

This project develops a machine-learning ranking system that uses content, search, traffic, engagement, freshness, and historical performance signals to prioritize pages associated with an observed downward trend.

A Random Forest classifier was evaluated using a client-grouped train/test split, with 23,837 training rows and 6,163 held-out test rows across 25 and 7 clients respectively.

On the held-out test set, the model achieved Precision@500 of 61.2%, compared with 26.0% for the baseline, representing an observed 2.35× lift.

The result is a directional decision-support tool for prioritizing human content review, not a causal model of Google's ranking system or a guarantee that refreshing a page will improve its future performance.

Introduction / Problem Statement

Which pages should we review first?

- **The decision**

Content teams can have many pages competing for limited editorial attention. Reviewing every page with the same priority is inefficient, while relying on a single performance metric can miss combinations of signals that indicate a page may deserve closer inspection.

- **This project asks:**

Can historical content, search, traffic, engagement, freshness, and performance signals be used to rank pages associated with an observed downward trend, so that a content team can prioritize human review?

- **What the system does**

The system takes an individual content item and produces a ranked review priority based on its estimated probability of belonging to the observed down trend category.

- **Each recommendation combines:**

[Model Score] → [Priority Rank] → [Reason Codes] → [Human Review]

The goal is not to automatically change or remove content. The goal is to help a content team decide where to look first.

- **Why machine learning?**

The dataset contains multiple interacting signals, including content characteristics, historical traffic, recent performance, freshness, search metrics, and engagement measures.

A machine-learning model provides a repeatable way to combine these signals into a ranking rather than relying on one manually selected threshold.

- **Decision boundary**

A high score means that a page resembles observations associated with the down category in the training data.

It does not mean that the page will definitely decline, that a specific edit will improve performance, or that the model has learned Google's ranking algorithm.

The output is therefore treated as directional decision-support for human review.

Data

- **Dataset**

The analysis uses the anonymized FlyRank ML starter dataset. The modeling dataset contains 30,000 content observations and 44 columns.

The data represents content-level search and performance information, including content characteristics, search demand, historical traffic, engagement, freshness, and performance indicators.

- **Unit of analysis**

The unit of analysis is an individual content item/page.

Each row represents one content observation with its associated search, traffic, engagement, and content attributes.

- **Target**

The target used for modeling is trend_direction.

- 1 = down
- 0 = other

In the full 30,000-row dataset:

Target	Observations	Share
Down	**16,262**	**54.2%**
Other	**13,738**	**45.8%**

The held-out test set had a 52.5% observed down base rate.

- **Feature data**

The final model used 41 features across several groups:

1. Search: search volume, competition, CPC
2. Content: content type, main intent, word count, character count
3. Historical performance: impressions, clicks, pageviews, sessions, users
4. Engagement: engaged sessions, scroll events, engagement rate, scroll rate
5. AI traffic: AI sessions and AI traffic percentage

6. Recent performance: last-30-day and previous-30-day metrics
7. Content age & freshness: content age, age tier, days since last update, freshness tier
8. Performance tiers: impression tier and position tier

- **What was excluded**

Several fields were deliberately excluded from the model.

Label-derived fields such as trend_direction and trend_pct were excluded because they directly describe the outcome and could cause target leakage.

Pseudonymous IDs such as client and content identifiers were not used as predictive features. Client identifiers were used only to create the grouped train/test split.

Any fields that could expose client identity, private queries, domains, URLs, or credentials were also kept out of the public analysis.

Validation split

To reduce the risk of learning client-specific patterns, the evaluation used a client-grouped split.

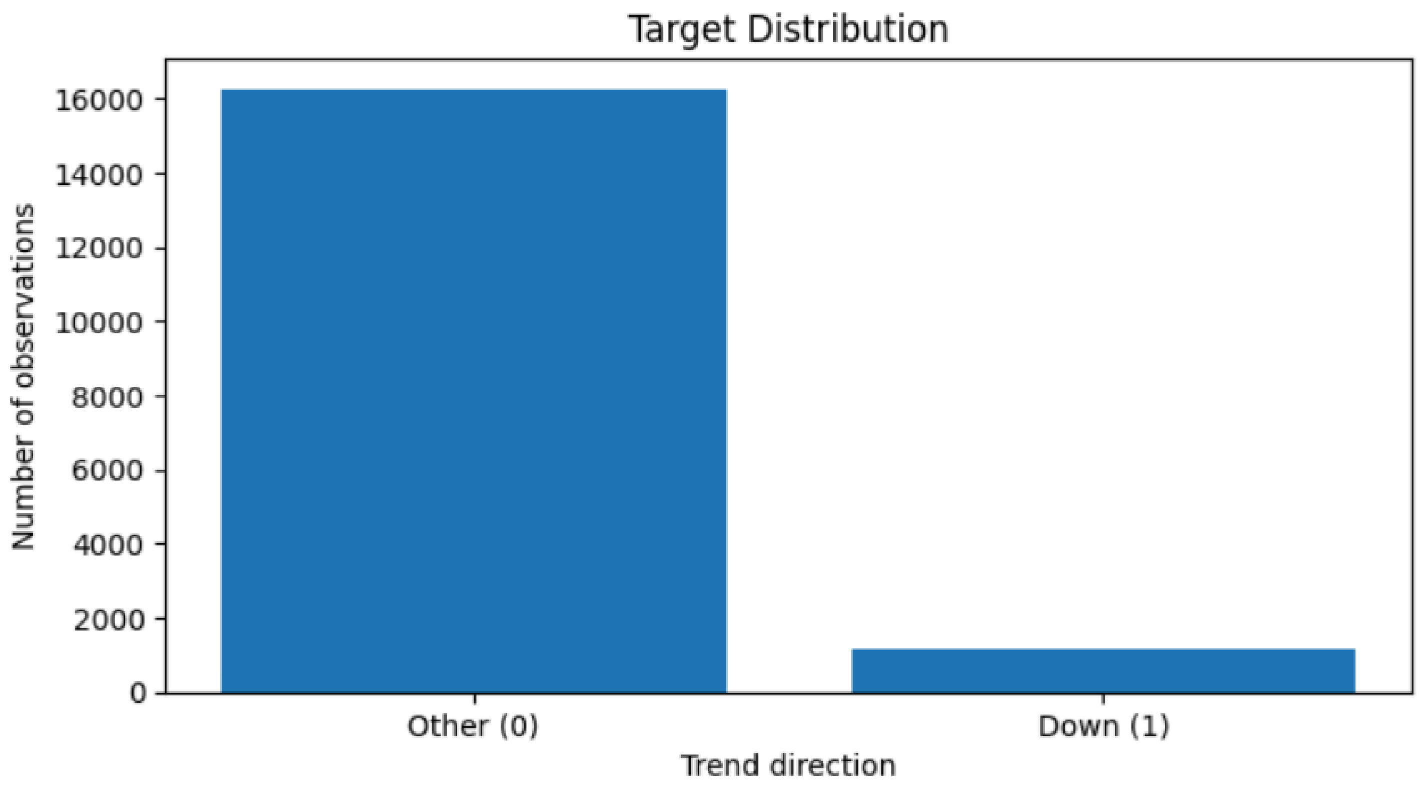
1. Training: 23,837 observations across 25 clients
2. Test: 6,163 observations across 7 clients
3. Client overlap: 0

This means the model was evaluated on clients that were not represented in its training data.

Public-safety rule

The paper reports only anonymized, aggregate information. No client names, domains, private queries, credentials, or raw client exports are included.

rows	**30000**
original columns	**44**
model features	**41**
held-out test rows	**6163**



Methodology

- **Approach**

The project uses a supervised machine-learning classification approach to rank content according to its likelihood of belonging to the observed down trend category.

The workflow was:

Data → Feature engineering → Leakage checks → Client-grouped split → Baseline → Random Forest → Precision@K → Ranked action queue

- **Feature engineering**

The original dataset was transformed into a final feature vector containing 41 model features.

The features combine:

1. Content characteristics
2. Search demand and competition
3. Historical traffic
4. Recent performance
5. Engagement signals
6. Content age
7. Freshness
8. Search-position indicators
9. Performance tiers

Categorical variables were encoded for modeling, while missing numerical and categorical values were handled during preprocessing.

- **Target definition**

The prediction target is:

trend_direction = 1 when the observed trend is down; otherwise 0.

This makes the task a binary classification problem.

- **Baseline**

A simple transparent baseline was evaluated before interpreting the machine-learning model.

The baseline was evaluated on the same held-out test set and with the same Precision@K metrics as the model.

This provides a direct reference for determining whether the model's ranking adds useful signal beyond the baseline.

- **Model**

A Random Forest classifier was used to estimate the probability that each content item belongs to the down category.

The predicted probabilities are then used to sort content items from highest to lowest priority.

The output is therefore not simply a yes/no prediction. It is a ranked review queue.

- ## Validation design

The evaluation uses a **client-grouped train/test split**.

Split	Observations	Clients
Training	**23,837**	**25**
Test	**6,163**	**7**

There is **zero client overlap** between training and test sets.

This design helps reduce the possibility that the model is evaluated on content from clients it effectively saw during training.

- **Leakage checks**

Before modeling, potential leakage sources were explicitly checked.

The following were excluded:

- trend_direction — the target itself
- trend_pct — label-derived information
- Pseudonymous identifiers as predictive features
- Future-window information that would not be available at prediction time

The resulting feature vector passed the leakage/exclusion check.

- **Evaluation metric**

The primary ranking metric is **Precision@K**.

For a selected K, Precision@K measures the proportion of the top K ranked content items that actually belong to the observed down category.

This metric matches the practical decision:

If a content team can only review K pages, how many of those pages are associated with the observed down category?

The model is therefore evaluated as a **prioritization system**, rather than only as a conventional binary classifier.

Results

- **Model performance**

The model was evaluated on the same held-out test set as the baseline using Precision@K.

K	Model	Baseline	Improvement	Lift
20	**0.550**	**0.250**	**+0.300**	**2.20×**
50	**0.480**	**0.260**	**+0.220**	**1.85×**
100	**0.470**	**0.230**	**+0.240**	**2.04×**
200	**0.510**	**0.230**	**+0.280**	**2.22×**
500	**0.612**	**0.260**	**+0.352**	**2.35×**
1000	**0.629**	**0.303**	**+0.326**	**2.08×**

The test-set down **base rate was 52.5%**. At K=500, the model achieved 61.2% Precision@500, compared with 26.0% for the baseline, representing a measured 2.35× lift over the baseline.

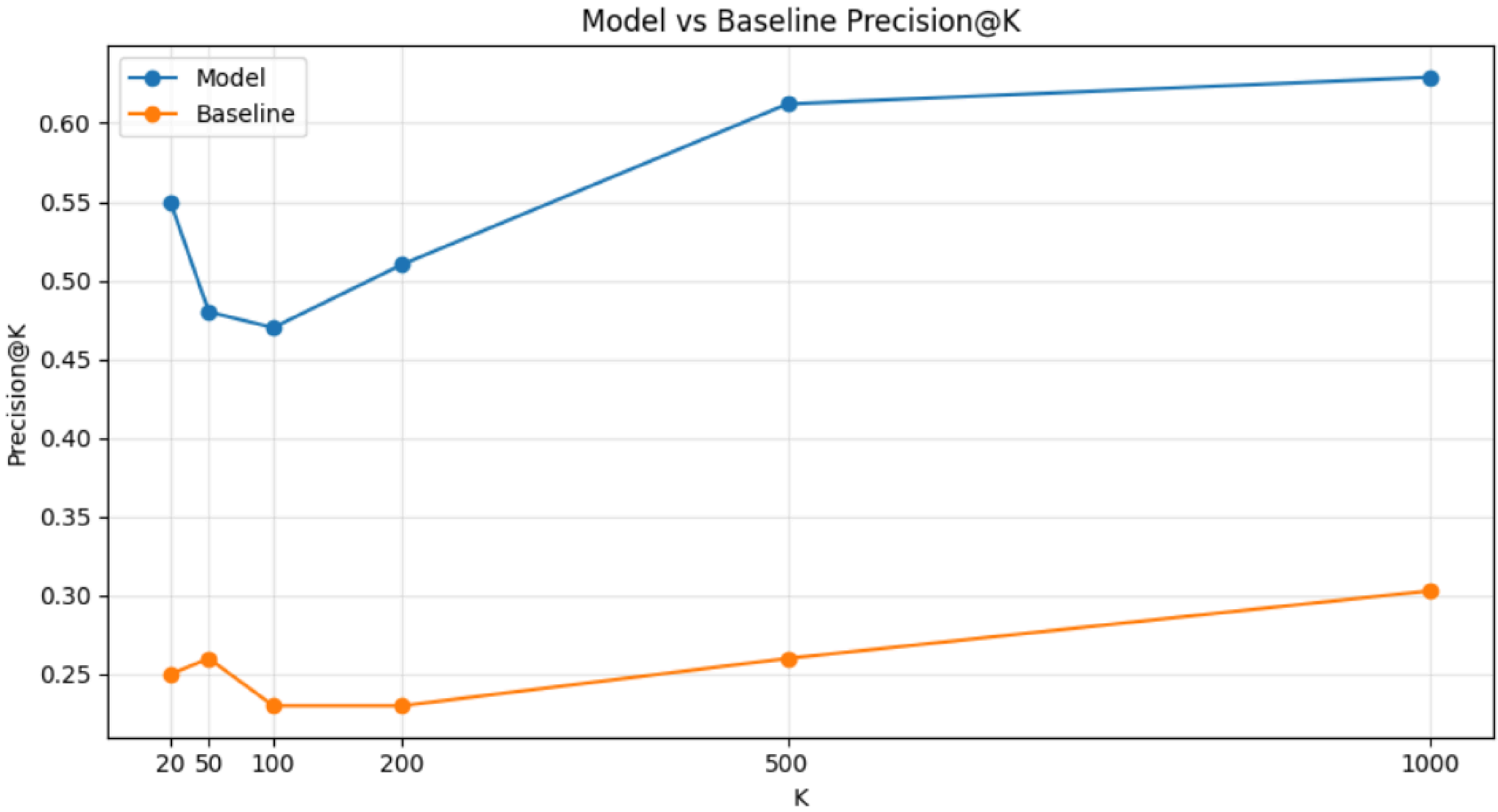


Fig. Model vs baseline Precision@K on the same held-out test set. The model shows higher precision than the baseline at every evaluated ranking depth, with the largest measured lift at K=500.

- **What the result shows**

The model consistently outperformed the baseline across all tested ranking depths. The strongest measured lift occurred at K=500, where approximately 61 out of every 100 items in the ranked set belonged to the observed down category, compared with approximately 26 out of 100 for the baseline.

These results indicate that the available features contain useful directional signal for prioritizing content associated with the observed down category. They do not establish that the model predicts Google's ranking algorithm or that any individual feature causes traffic or ranking changes.

Limitations & Interpretation

- **What the model found**

The feature-importance analysis shows that **position_tier** was the strongest individual feature in the trained model, followed by **freshness_tier, word_count**, **content_age_days**, and **char_count.**

Other relatively important signals included recent sessions, age-related features, days since the last update, model/provider information, and recent clicks.

These importance values indicate which features contributed most to the model's decisions. They do not demonstrate causation. A feature being important does not mean changing that feature will necessarily improve or worsen search performance.

Rank	Feature	Importance
1	position_tier	0.1203
2	freshness_tier	0.0882
3	word_count	0.0762
4	content_age_days	0.0639
5	char_count	0.0622
6	sessions_last_30d	0.0620
7	age_tier	0.0536
8	days_since_last_update	0.0507
9	model_used	0.0463
10	clicks_prev_30d	0.0392

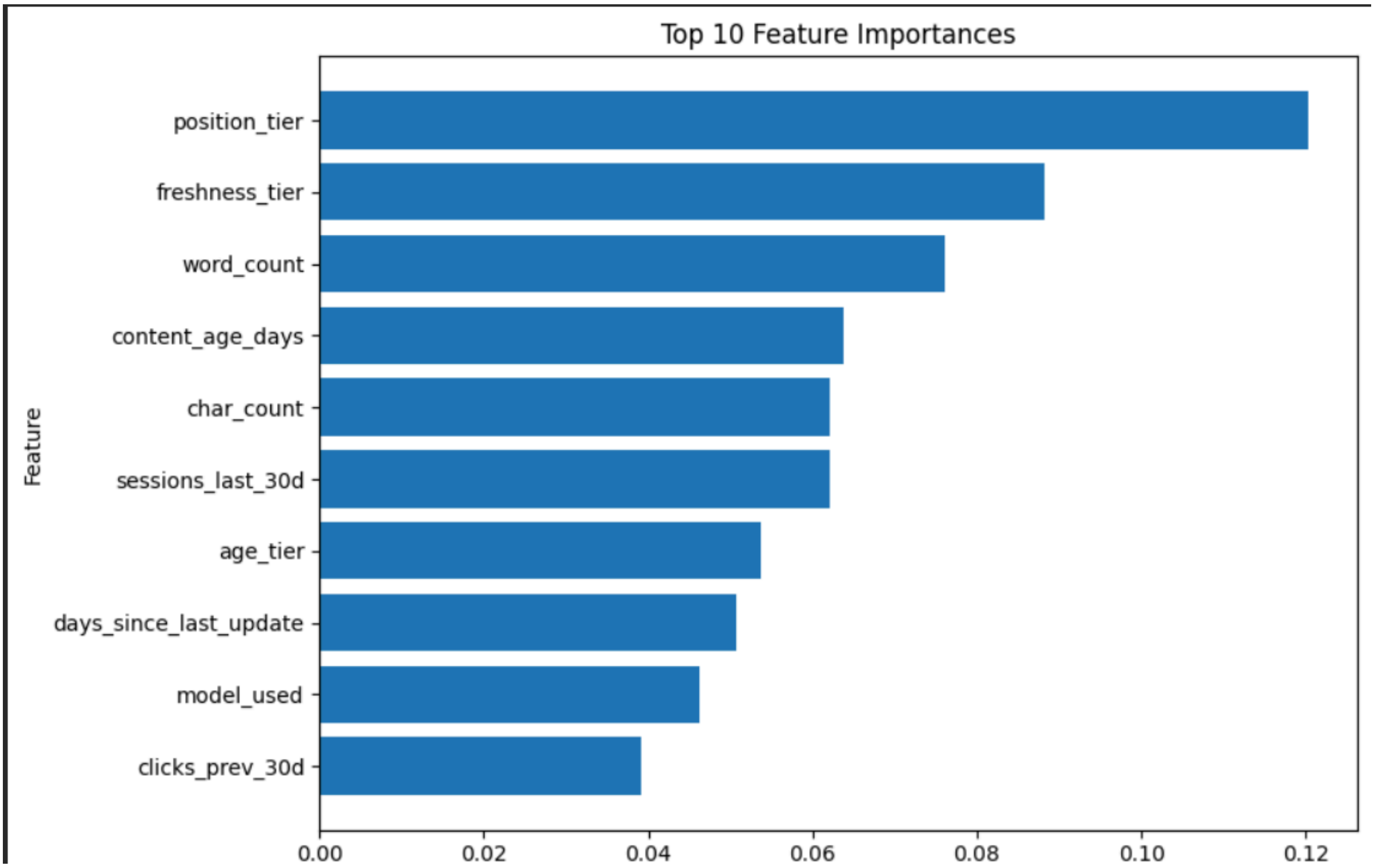


Fig. Top 10 feature importances from the trained model. Position tier and freshness-related features had the highest measured importance. These values describe model associations and should not be interpreted as causal effects.

- **Limitations**

This analysis has several important limitations:

1. The target represents an observed trend category, not Google's ranking algorithm.
2. Feature importance shows association with model decisions, not causal effects.
3. The evaluation uses a held-out client-grouped split, but the results do not guarantee performance on every future client or dataset.
4. Precision@K depends on the chosen ranking depth and should be interpreted alongside the 52.5% test-set base rate.
5. Historical performance variables may reflect existing differences between content items rather than factors that directly cause future changes.
6. The model should therefore be treated as a decision-support and prioritization tool, not an automated decision-maker.

Ranked recommendations

The model output is converted into a ranked review queue using the predicted probability of the observed down category.

Each content item receives a priority rank, a model-estimated probability, and one or more reason codes. The reason codes provide a simple explanation for why an item was placed in the review queue.

The current action rules identify three main situations:

1. **DECLINE_RISK_HIGH — model-estimated probability of at least 0.70 for the observed down category.**
2. **CONTENT_STALE — more than 180 days since the last update.**
3. **RECENT_PERFORMANCE_WEAK — impressions in the last 30 days are lower than the previous 30-day period.**
4. **MODEL_REVIEW — used when none of the specific rule-based reasons are triggered.**

How a content team could use the queue

The highest-ranked items can be reviewed first rather than treating the model output as an automatic decision. Editors can inspect the underlying content and decide whether the appropriate action is to refresh, improve, monitor, or leave the content unchanged.

The ranking is therefore intended as a decision-support mechanism for prioritizing human review, not as an instruction to make a specific change.

```
Top 20 ranked recommendations:
```

	priority_rank	content_id	predicted_probability	reason_codes	trend_direction
0	1	content_643b51c0a848	0.521638	RECENT_PERFORMANCE_WEAK	down
1	2	content_25ff6043653f	0.519480	MODEL_REVIEW	stable
2	3	content_676074f0d01d	0.519480	MODEL_REVIEW	up
3	4	content_bfd5fbed5749	0.519480	MODEL_REVIEW	up
4	5	content_96fa31b9f575	0.519480	RECENT_PERFORMANCE_WEAK	down
5	6	content_720edd4e1491	0.518820	MODEL_REVIEW	up
6	7	content_1980254ef582	0.517989	RECENT_PERFORMANCE_WEAK	down
7	8	content_f697fc3a64e3	0.517978	RECENT_PERFORMANCE_WEAK	down
8	9	content_83784080ee4d	0.517631	MODEL_REVIEW	stable
9	10	content_fbbacf50108d	0.517631	RECENT_PERFORMANCE_WEAK	down
10	11	content_f73c382ac270	0.517349	RECENT_PERFORMANCE_WEAK	down
11	12	content_6607847e4454	0.517300	RECENT_PERFORMANCE_WEAK	down
12	13	content_ce59581533ca	0.516941	RECENT_PERFORMANCE_WEAK	stable
13	14	content_54af84f1e737	0.516552	RECENT_PERFORMANCE_WEAK	stable
14	15	content_e6c0c4e9ae59	0.516487	RECENT_PERFORMANCE_WEAK	down
15	16	content_cf9f8ecdf9de	0.516373	RECENT_PERFORMANCE_WEAK	down
16	17	content_68815fc0cee8	0.516373	RECENT_PERFORMANCE_WEAK	down
17	18	content_4d9f36001f06	0.516312	RECENT_PERFORMANCE_WEAK	stable
18	19	content_3395aba722a2	0.516312	RECENT_PERFORMANCE_WEAK	down
19	20	content_f6a8bb38f970	0.516155	MODEL_REVIEW	up

Fig. Top 20 ranked content review queue. Items are ordered by model-estimated probability of the observed down category and accompanied by reason codes to support human review.

- ## Confidence and limits

The ranking reflects patterns learned from the available data and should be interpreted as **directional**. A high model probability does not prove that a page will decline, nor does a reason code establish that the associated factor caused the observed trend.

Reproducibility & Acknowledgments

- **Reproducibility**

The analysis is organized as a sequence of weekly notebooks followed by the final capstone notebook.

The repository contains:

- **work/notebooks/w01_research_question.ipynb**
- **work/notebooks/w02_ml_task_framing.ipynb**
- **work/notebooks/w03_data_contract.ipynb**
- **work/notebooks/w03_feature_leakage_check.ipynb**
- **work/notebooks/w04_baseline_score.ipynb**
- **work/notebooks/w04_signal_audit.ipynb**
- **work/notebooks/w05_model.ipynb**
- **work/notebooks/w06_validation_audit.ipynb**
- **work/notebooks/w07_action_playbook.ipynb**
- **work/notebooks/capstone.ipynb**

The capstone notebook contains the final feature construction, grouped validation, model training, baseline comparison, feature-importance analysis, and ranked recommendation output.

The analysis was developed in Python using standard data-science and machine-learning tooling. Randomized modeling steps use fixed random seeds where applicable to support repeatability.

To reproduce the analysis, a user should:

1. Clone the repository.
2. Install the required Python dependencies.
3. Obtain the permitted FlyRank dataset access.
4. Run the notebooks in order.
5. Run capstone.ipynb from top to bottom.

The reported results should be regenerated from the same dataset release and preprocessing pipeline rather than copied manually.

- **Repository**

The repository contains the notebooks and supporting capstone artifacts required to reproduce the analysis.

The deployed research paper URL is stored separately in:

submission/paper_url.txt

- **Acknowledgments & Data Credit**

This work was built as part of the **FlyRank ML Internship capstone** using the anonymized FlyRank ML Internship dataset.

Built on the FlyRank ML Internship dataset.

Data credit: FlyRank

- **Research framing**

The results in this paper are presented as observed, measured, and directional evidence for decision support. They should not be interpreted as causal findings or as evidence about Google's ranking algorithm.